EXPLOITS UNIVERSITY

BSc INFORMATION COMMUNICATION AND TECHNOLOGY

BICT 3202 · OBJECT-ORIENTED ANALYSIS AND DESIGN

WEEK 7

Object-Oriented Analysis — Dynamic Modelling

PROGRAMME	BSc Information Communication and Technology
COURSE CODE / YEAR	BICT 3202 · Year 3
LECTURER	Francis Fweta — ICT Lecturer
DELIVERY	2h Lecture · 1h Tutorial · 1h Practical · 10h Independent Learning

Prepared by Francis Fweta · Exploits University · Week 7 of 16

Contents

1. Week 7 Introduction

Previous weeks mainly asked: **"What things exist in the system and how are they structurally related?"** Week 7 asks a different question: **"What happens to those things over time, and how do they respond to events?"** That distinction is fundamental to object-oriented modelling. The OMG UML specification separates UML into structural and behavioural semantic areas — state machines, activities and interactions are behavioural modelling constructs, while classes, classifiers and packages belong to structural modelling.

This week's official topics are: **event, state, operations, concurrency, the relation of structure and dynamic models, and CASE tool support** — all six are covered, not simply "state diagrams".

2. Learning Outcomes

By the end of this week, a student should be able to:

- Define dynamic modelling and explain why dynamic behaviour must be modelled.
- Define an event and identify different types of events.
- Define a state and distinguish a state from an event.
- Explain state transitions, initial and final states, and guards.
- Explain entry, exit and do-activity behaviour.
- Define an operation and distinguish an operation from an event.
- Construct a UML state-machine diagram.
- Explain concurrency and distinguish sequential from concurrent behaviour.
- Explain the relationship between structural and dynamic models.
- Use a CASE tool to create and inspect dynamic models.

PART A — WHAT IS DYNAMIC MODELLING?

3. What Does "Dynamic" Mean?

Dynamic means **something that changes over time**. In a student registration system, a student moves from **Not Registered** to **Registration Pending** (after submitting) to **Registered** (after payment is confirmed). That changing behaviour is what dynamic modelling describes.

Structural modelling asks "what exists?" — Student, Course, Lecturer, Registration — and how they relate. **Dynamic modelling asks "what happens?"** — Student selects course, registration is submitted, payment is made, registration is confirmed. The formal specification describes behavioural semantics in terms of behaviour and state changes built upon the structural model.

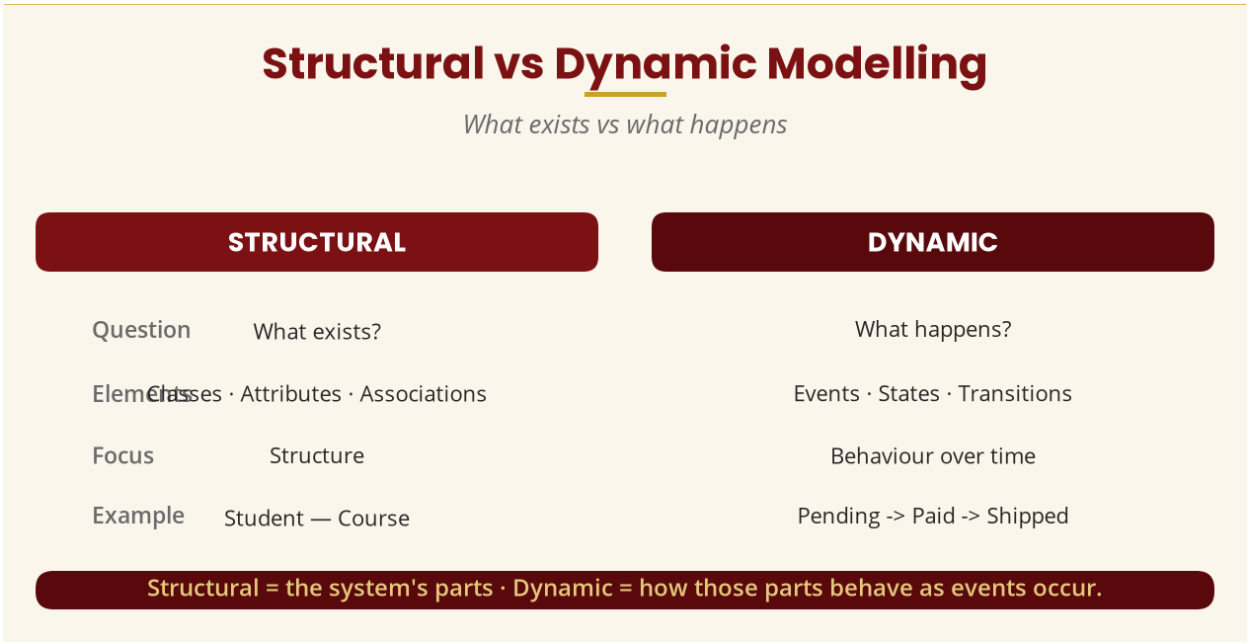

Figure 1 — Structural vs dynamic modelling

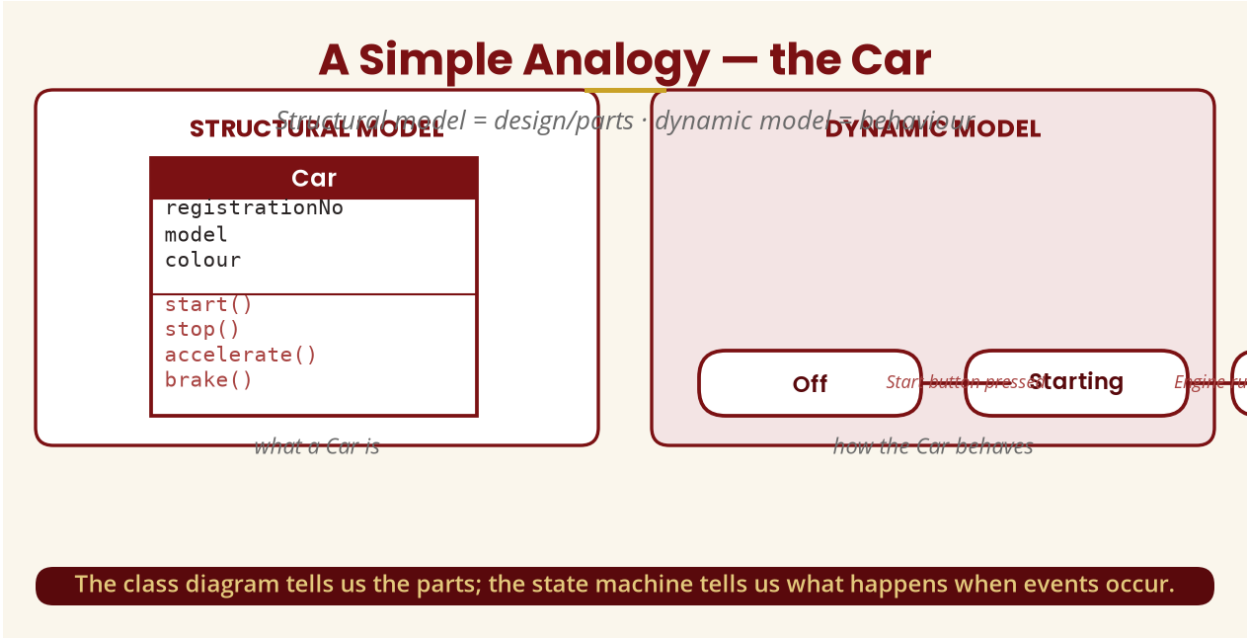

Figure 2 — The car analogy: design/parts vs behaviour

PART B — EVENTS

4. What Is an Event?

An **event** is something that occurs and can cause a change in behaviour or state — in simple English, **something that happens to which the system can respond**. Examples: a user clicks Login, a customer places an Order, an ATM card is inserted, a payment is received, a temperature exceeds a limit, a student submits an assignment.

Events fall into broad categories: **user-generated** (click Login, submit form, select product, upload file), **system-generated** (timer expires, backup completed, session timeout), **external**

(payment gateway confirmation, bank transaction response, sensor detects movement), and **data-related** (payment received, stock reaches zero, balance falls below threshold).

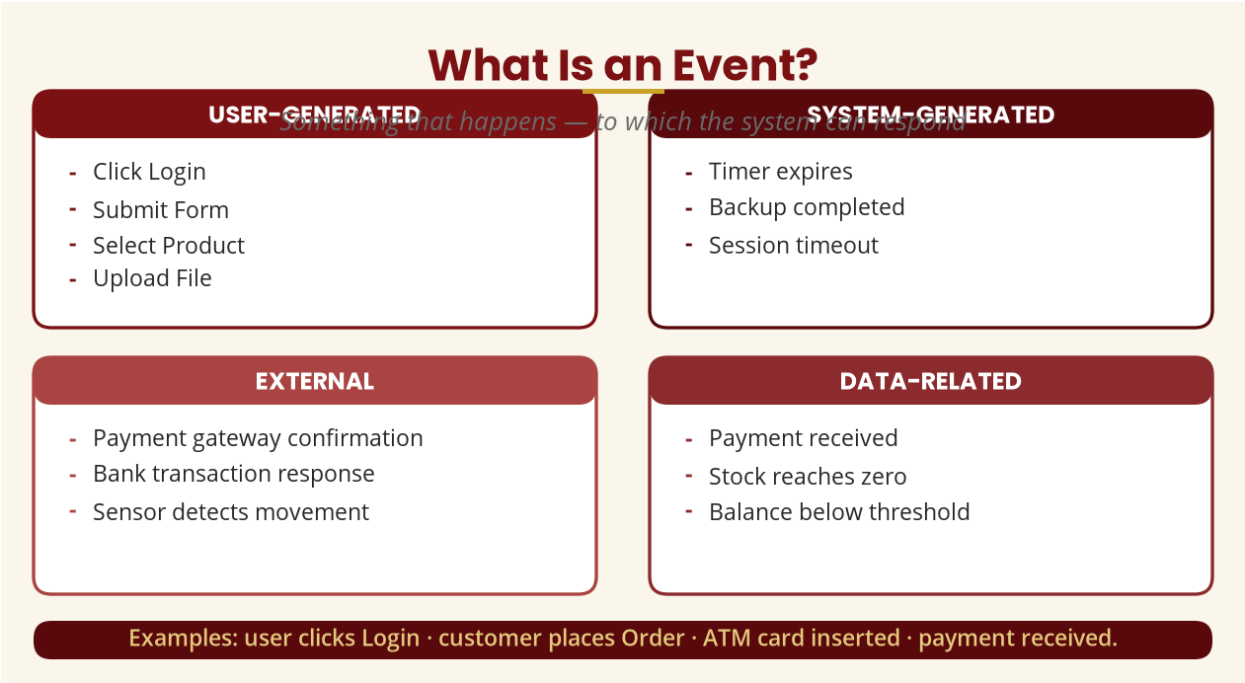

Figure 3 — Categories and examples of events

5. Events Are NOT States

This is a common student mistake. An **event** is something that happens — "Customer clicks Pay". A **state** is a condition that exists for some period of time — "Payment Pending". So: event something happens; state something is/exists. An event can trigger a **transition**: in `[Locked] –Correct PIN→ [Unlocked]`, Locked and Unlocked are states and "Correct PIN" is the event/trigger.

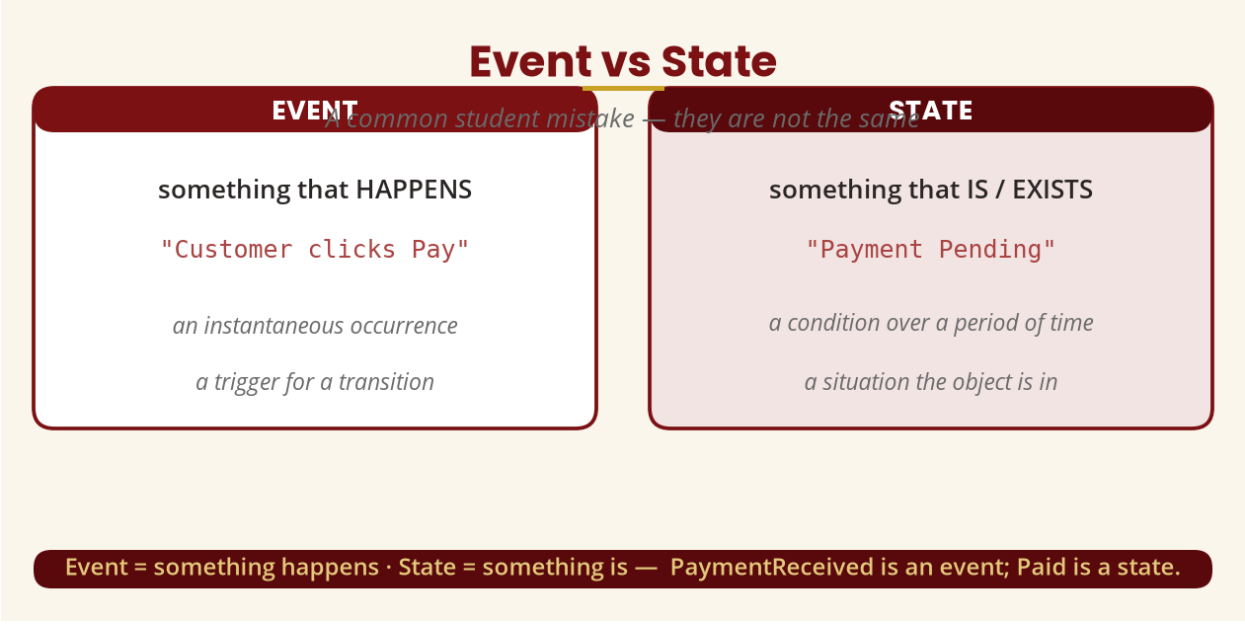

Figure 4 — Event vs state

PART C — STATES

6. States and the UML State Diagram

A **state** represents a condition or situation in which an object exists during some period of its behaviour. An Order can be Pending, Paid, Processing, Shipped, Delivered or Cancelled. In a state-machine diagram, the **filled circle** (●) is the initial pseudostate, the **bull's-eye** (◉) is the final state, rounded rectangles are states, and labelled arrows are transitions carrying the trigger/event.

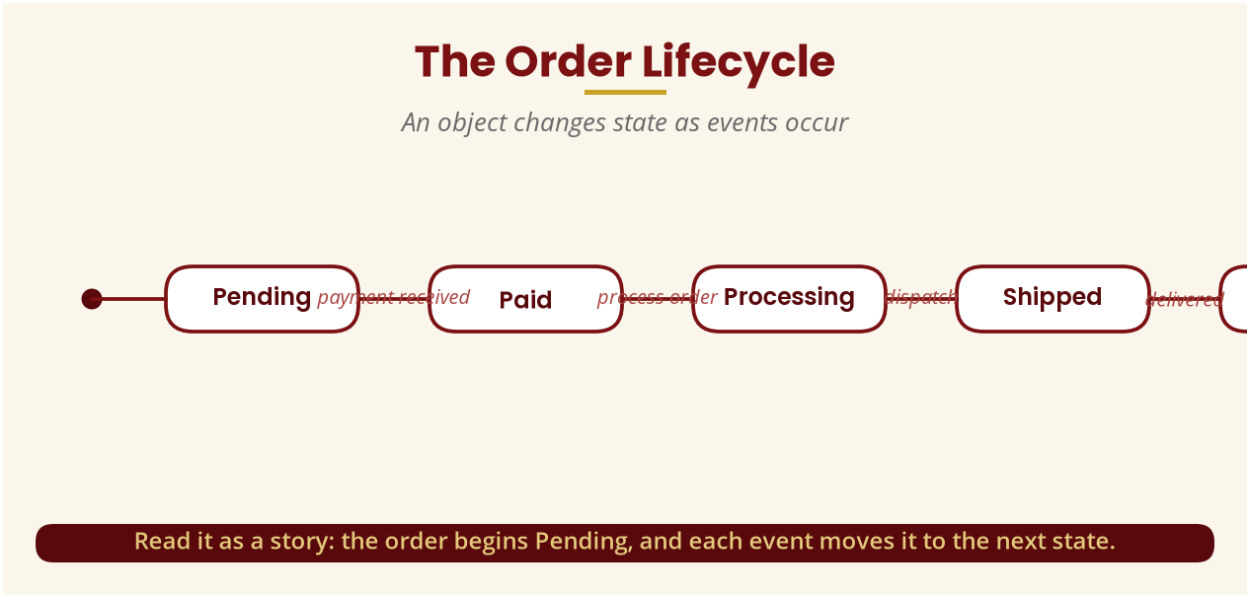

Figure 5 — The Order lifecycle

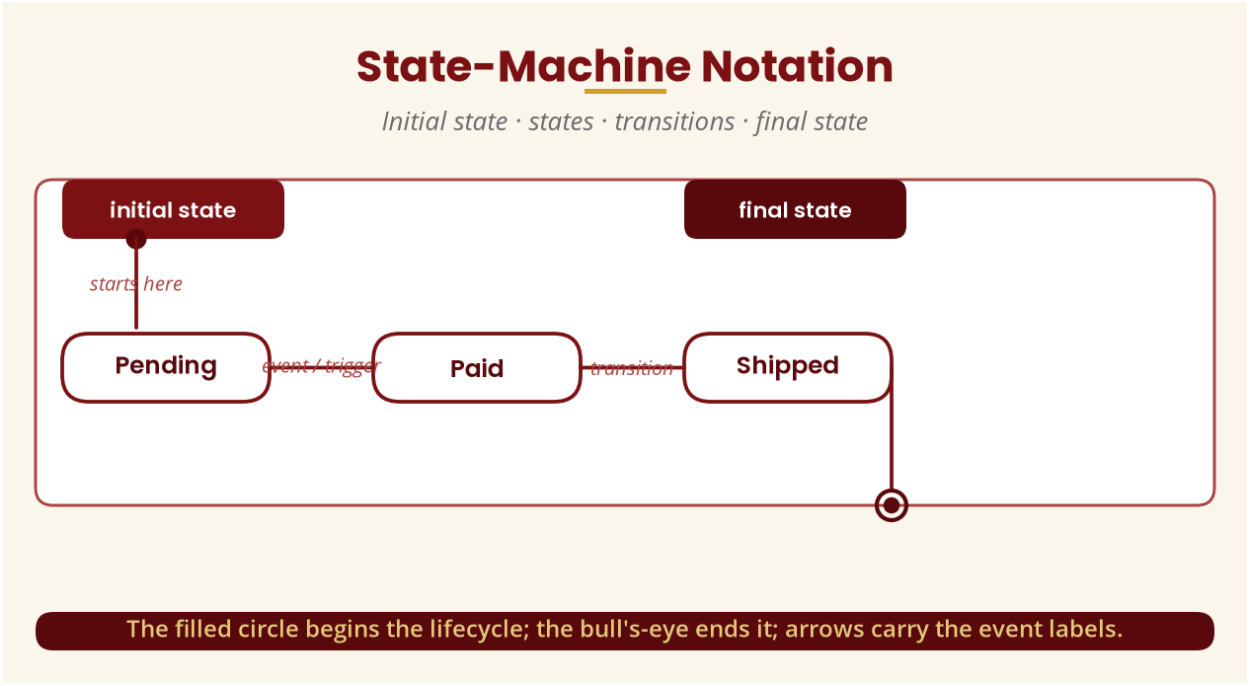

Figure 6 — State-machine notation

7. Transitions and Guards

A **transition** describes movement from a source state to a target state — `Pending` —`payment received`→ `Paid`. Sometimes an event alone is not enough: the system may need a **condition**. A **guard** is written in square brackets, e.g. `payment received [amount ≥ total]`, and means the transition can happen only if the condition is true. One state can have several possible transitions — an ATM withdrawal with sufficient balance dispenses cash, while insufficient balance leads to "Insufficient Funds".

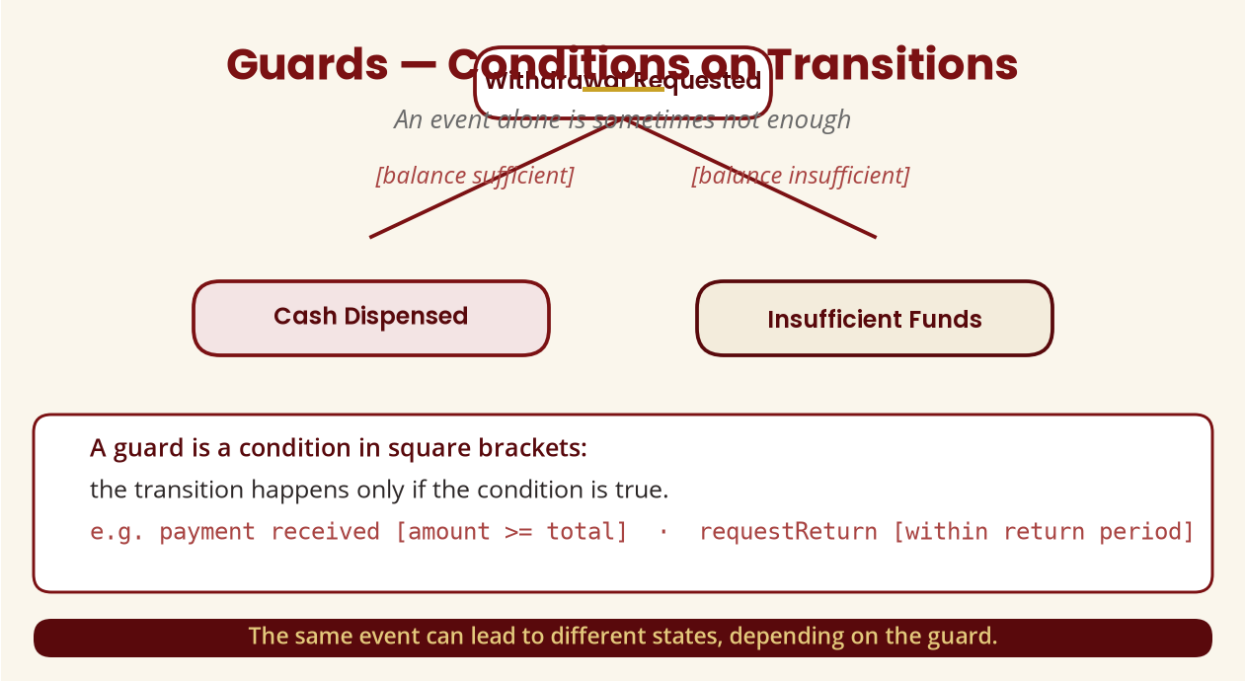

Figure 7 — Guards on transitions (ATM withdrawal)

8. Entry, Exit and Do Behaviour

A state can carry behaviour: **entry** — action performed when the state is entered (`entry / sendConfirmationEmail()`); **exit** — action performed when the state is left (`exit / updateTransactionLog()`); and **do** — behaviour that occurs while the object remains in the state (`do / processOrder()`), which is different from an instantaneous transition trigger. The UML specification recognises entry, exit and do-activity behaviours associated with states.

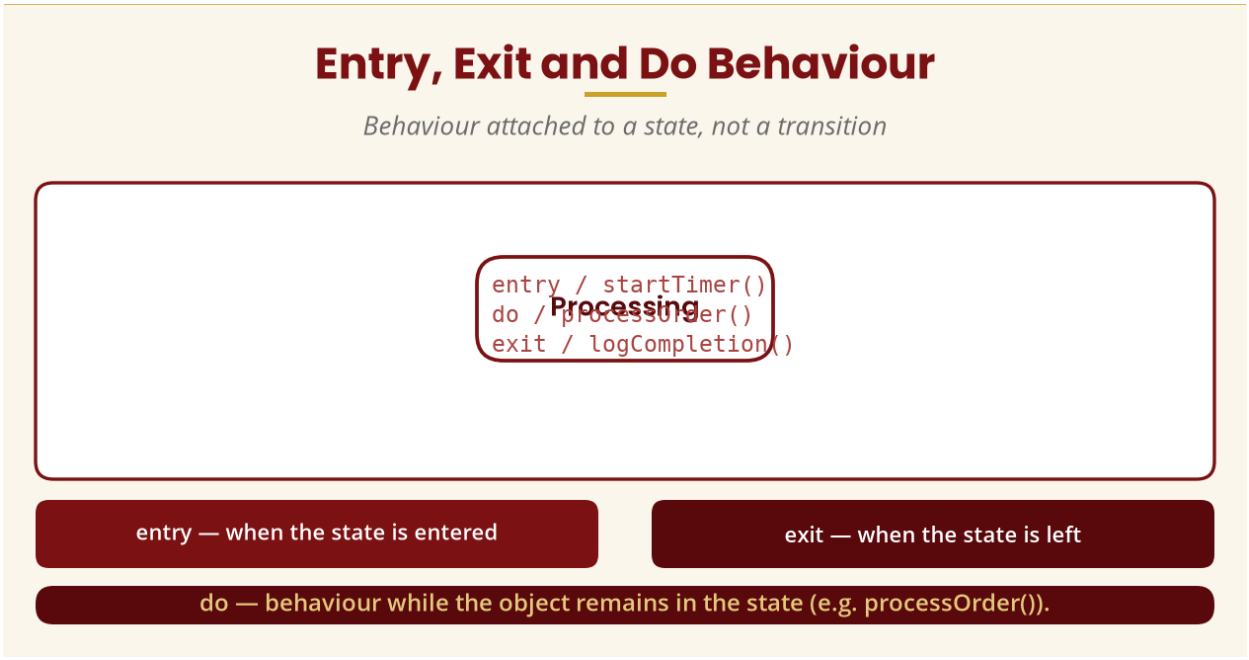

Figure 8 — Entry, exit and do behaviour

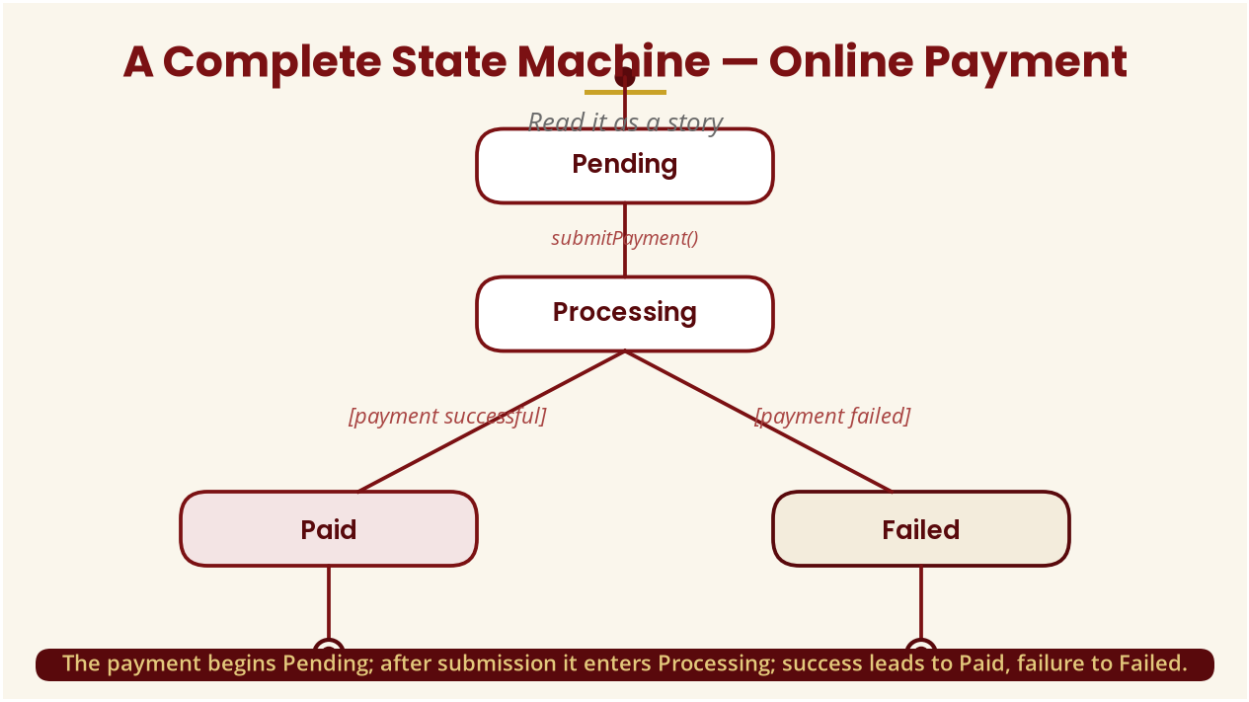

Figure 9 — A complete state machine (online payment)

PART D — OPERATIONS

9. Operations and State Change

An **operation** is a service or behaviour that an object/class provides — Order has calculateTotal(), submitOrder() and cancelOrder(). Students often confuse operations with events: an operation is something the object can **perform or provide** (it can be invoked), while an event is something that **occurs** and can trigger behaviour. Operations can participate in dynamic behaviour — invoking `insertCard()` moves an ATM from Idle to Card Inserted, and `ejectCard()` returns it to Idle.

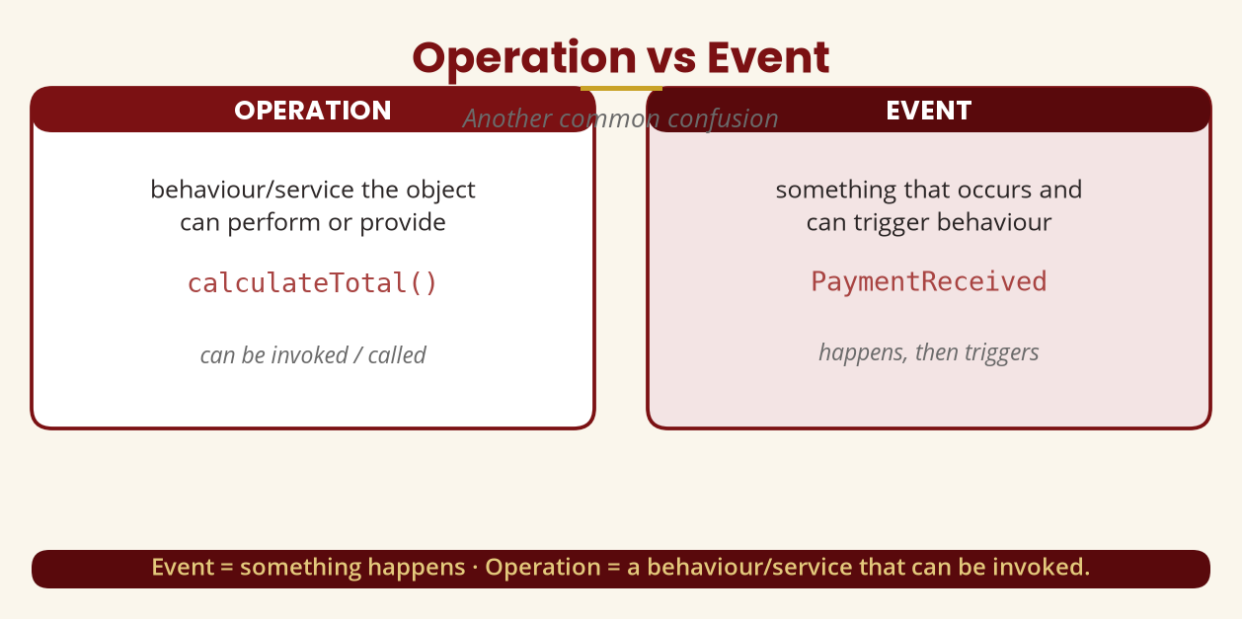

Figure 10 — Operation vs event

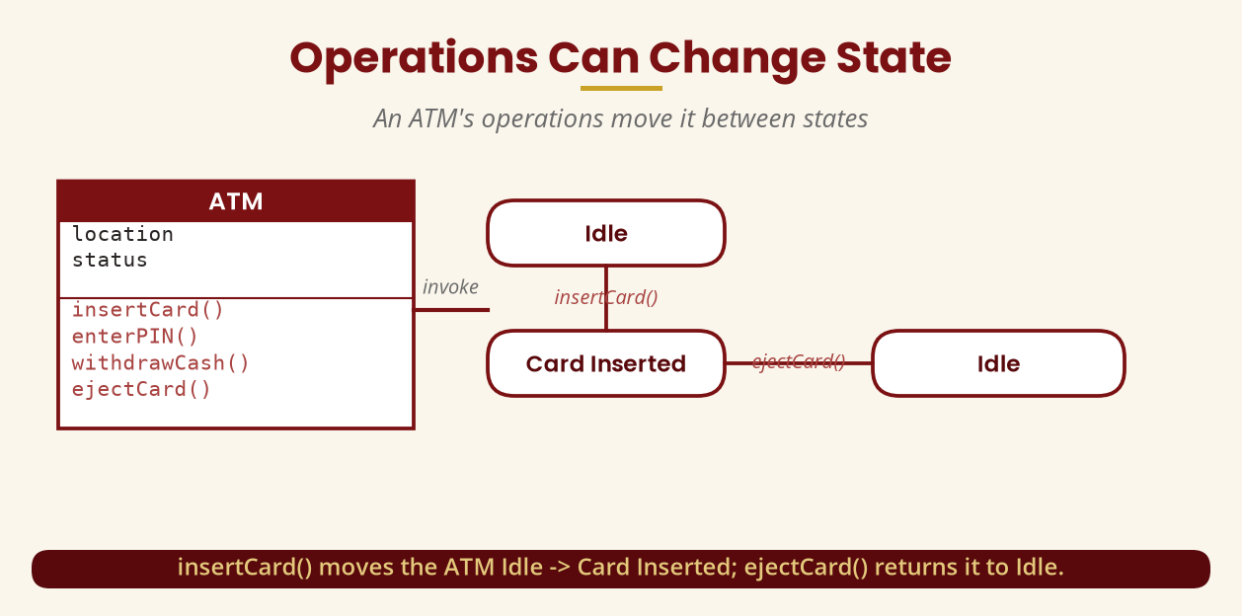

Figure 11 — Operations causing state change

PART E — CONCURRENCY

10. Concurrency

Concurrency means multiple activities or behaviours can occur during the same period rather than strictly one after another. After placing a food order, the restaurant prepares food **while** a driver is assigned **while** payment is processed — they need not happen in a strict sequence. **Sequential** behaviour runs one thing after another (Enter PIN Validate PIN Display account); **concurrent** behaviour runs activities in parallel (Order Placed splits into Process Payment and Prepare Order).

A real-life example: when a passenger checks in at an airport, security screening and baggage processing can happen concurrently, then both complete before boarding. In UML, a composite state can contain **concurrent regions** — Order Processing can split into a Payment region and a Delivery region that execute concurrently and join when both finish. Modern systems — banking, e-commerce, social media, mobile apps — routinely perform several activities simultaneously, so analysts must understand non-sequential behaviour.

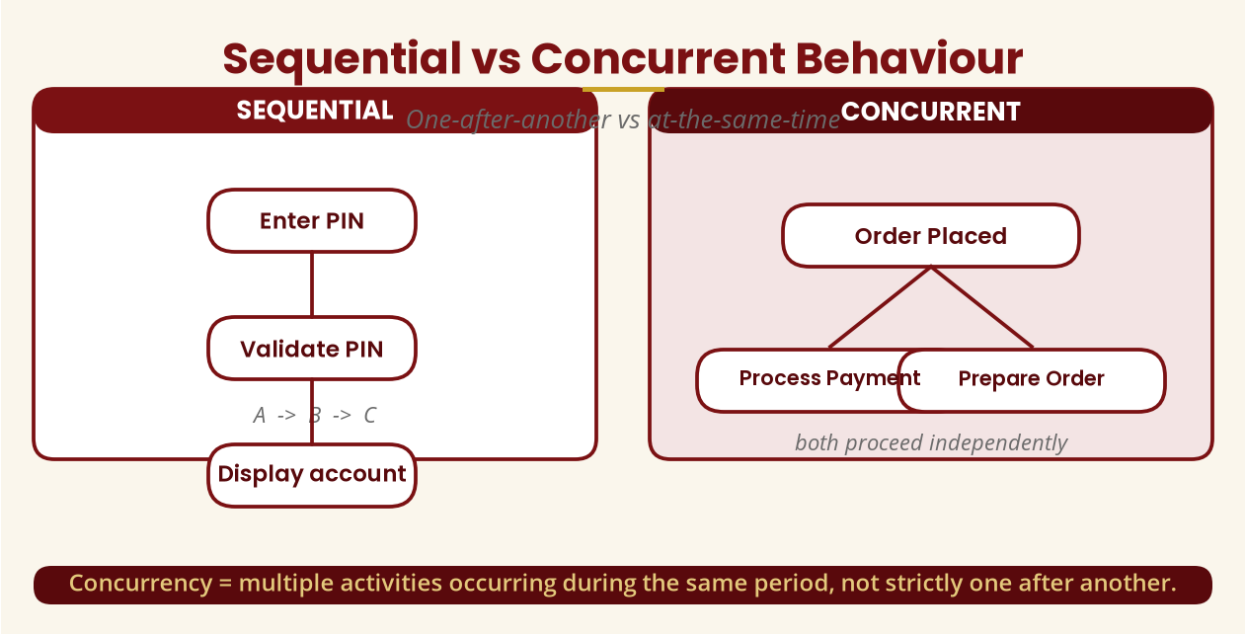

Figure 12 — Sequential vs concurrent behaviour

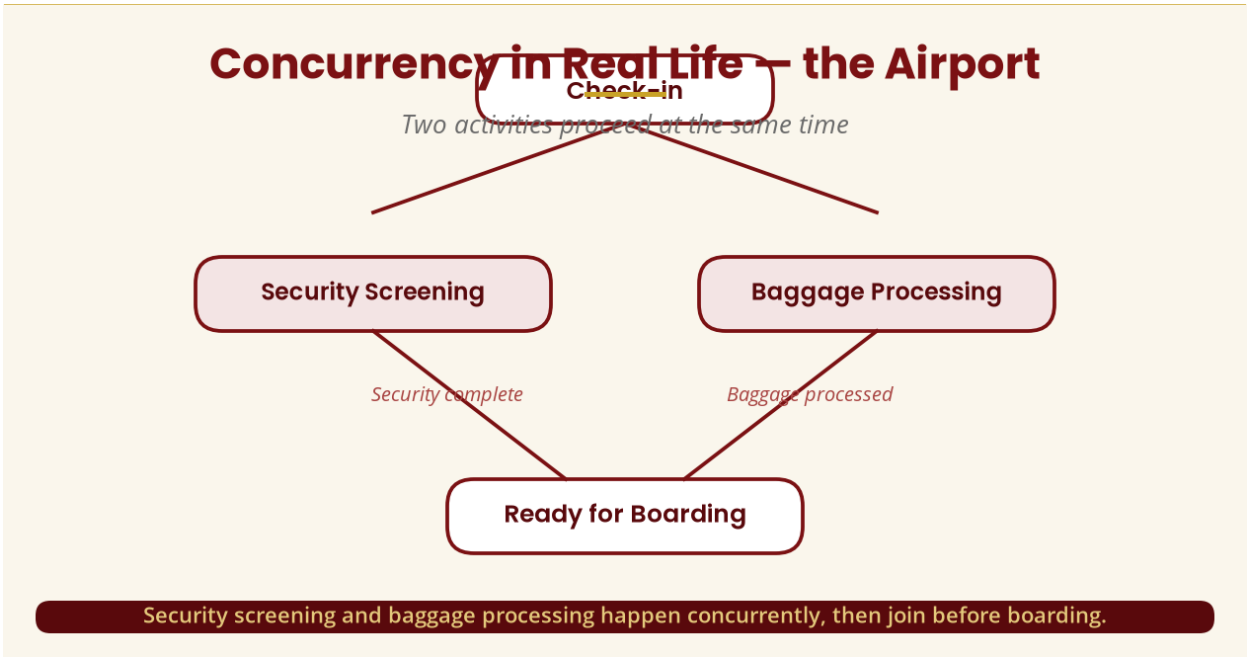

Figure 13 — Concurrency at the airport

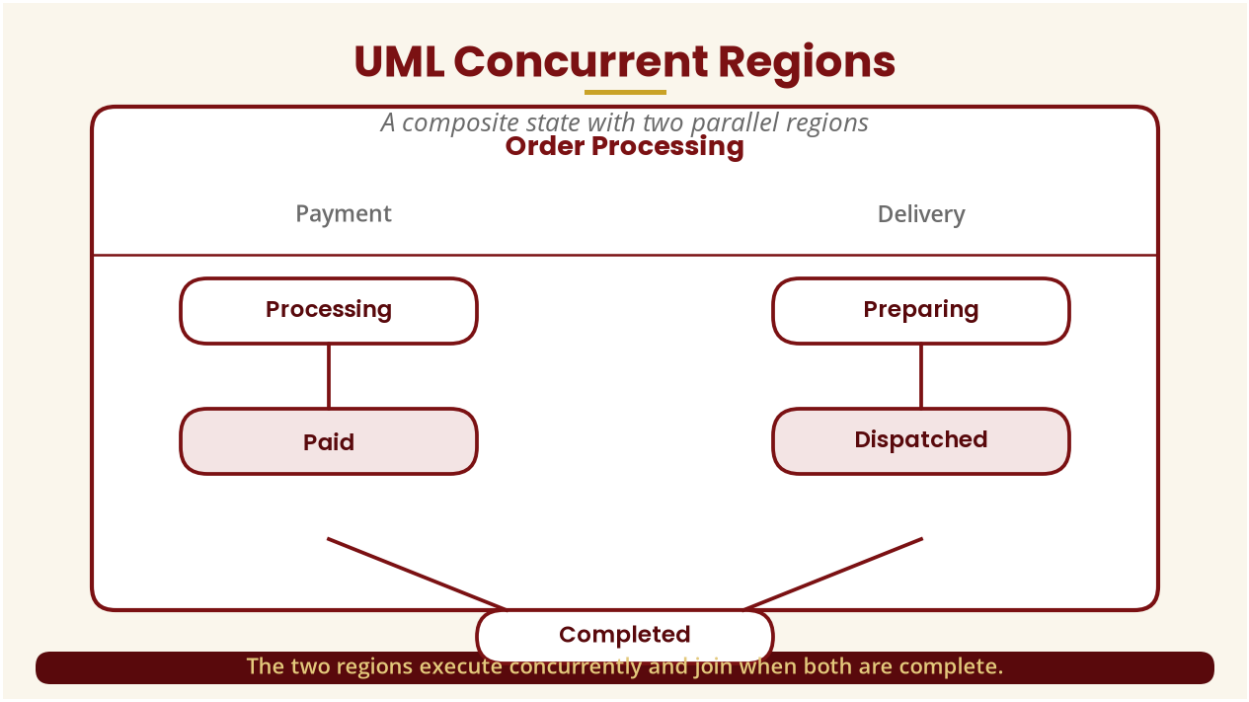

Figure 14 — UML concurrent regions

PART F — RELATIONSHIP BETWEEN STRUCTURAL AND DYNAMIC MODELS

11. Why Do We Need Both?

A class diagram tells us the structure of an Order, but not what sequence of behaviour is allowed — can an order go directly from Pending to Shipped without payment? The dynamic model makes such rules visible. **Structural model** = "what is the system made of?"; **dynamic model** = "how does the system behave?"; together they give **complete understanding**. Neither replaces the other.

Dynamic models can also **reveal requirements problems**: if the model shows a Delivered order moving to Cancelled, that may not make business sense — perhaps the correct rule is that Pending, Paid and Processing orders can be cancelled, while Shipped and Delivered orders enter a Return process. That is an important reason analysts model behaviour.

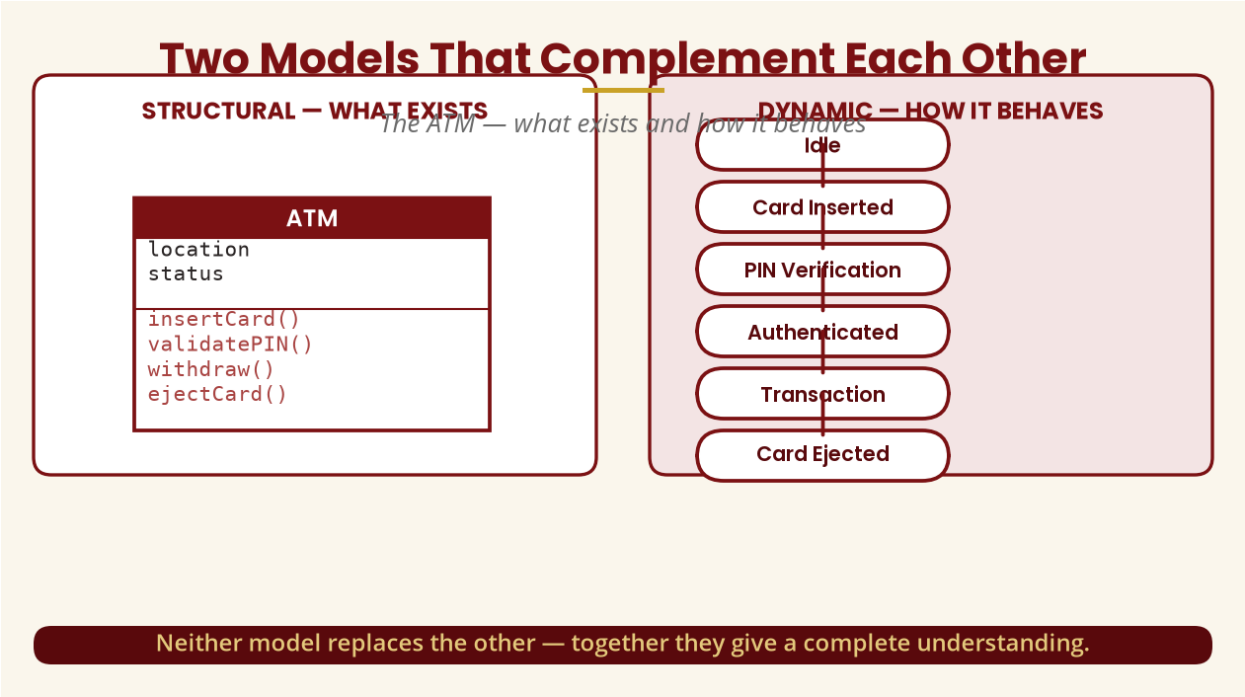

Figure 15 — ATM: structural and dynamic models together

PART G — CASE TOOL SUPPORT

12. Using a CASE Tool for Dynamic Models

CASE (**Computer-Aided Software Engineering**) tools help analysts create class, state-machine, activity, sequence, use-case and package diagrams. When a lecturer says “change Customer to User”, a CASE tool makes updating 30 classes and 20 transitions far easier than manual drawing. Examples include Enterprise Architect, Visual Paradigm, IBM Rational tools, StarUML, PlantUML and diagrams.net/draw.io; the course outline specifies a Select Enterprise demonstration. Students should be able to create a project, add states, transitions, event labels, guards, entry/exit behaviour and final states, then modify the model when requirements change.

PART H — COMPLETE EXAMPLE

13. Hospital Patient Dynamic Model

Model a patient appointment with states Appointment Requested, Confirmed, Checked In, Waiting, With Doctor, Completed and Cancelled, and events requestAppointment, confirmAppointment, patientArrives, joinQueue, doctorCallsPatient, consultationComplete and cancelAppointment. **Read the diagram as a story**: an appointment begins Requested; when confirmed it moves to Confirmed; when the patient arrives it becomes Checked In; the patient waits until the doctor calls them; the appointment enters With Doctor; when the consultation finishes it becomes Completed — or it can be cancelled before completion. If a student cannot explain the diagram in words, they probably do not understand the model.

Add guards where business rules apply — `confirmAppointment [doctor available]` leads to Confirmed, while `[doctor unavailable]` leads to Waitlisted. Add operations to the Appointment class (request(), confirm(), cancel(), checkIn(), complete()) — the dynamic model explains **when** those behaviours are allowed.

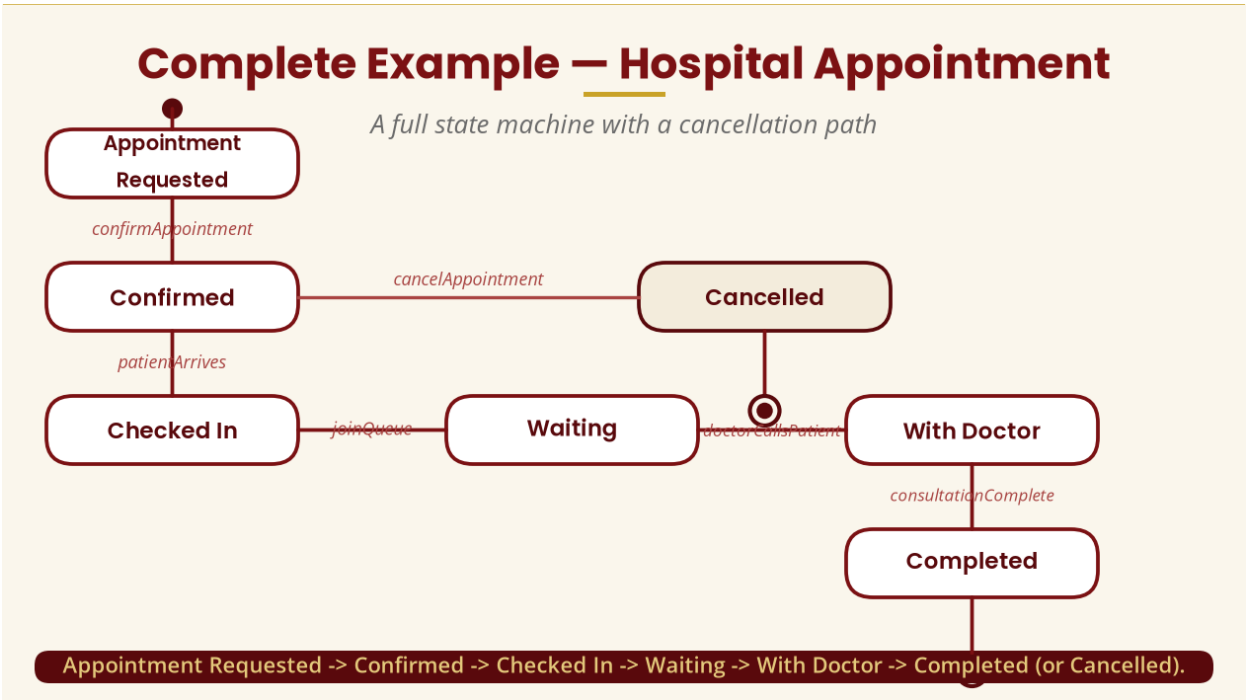

Figure 16 — Hospital appointment state machine

14. State vs Class vs Object — and State Machine vs Flowchart

Class is a blueprint/type (Order); **object** is a specific instance (order1024); **state** is the condition of that object (Paid); **event** is something that happens (PaymentReceived). Different objects of the same class can be in different states: Order #1001 □ Paid, #1002 □ Pending, #1003 □ Shipped. **A state describes the current condition of an object during its lifecycle.**

A state machine is **not** a flowchart: a flowchart represents procedural flow (calculate total □ check payment □ print receipt), while a state machine focuses on the states of an entity and the transitions between them in response to events. Dynamic modelling is especially useful when an object has several meaningful states, changes state during its lifecycle, responds to events, has business rules governing transitions, or has concurrent behaviour — Order, BankAccount, ATM, TrafficLight, PatientAppointment, LoanApplication, Booking, InsuranceClaim, StudentRegistration.

15. The Traffic Light Example

The traffic light is excellent for beginners because the states are obvious: Red, Green, Yellow — timer expiry drives each transition. A modern intersection has a North/South controller and an East/West controller whose behaviour must coordinate, introducing concurrent state-machine regions whose transitions must be coordinated safely.

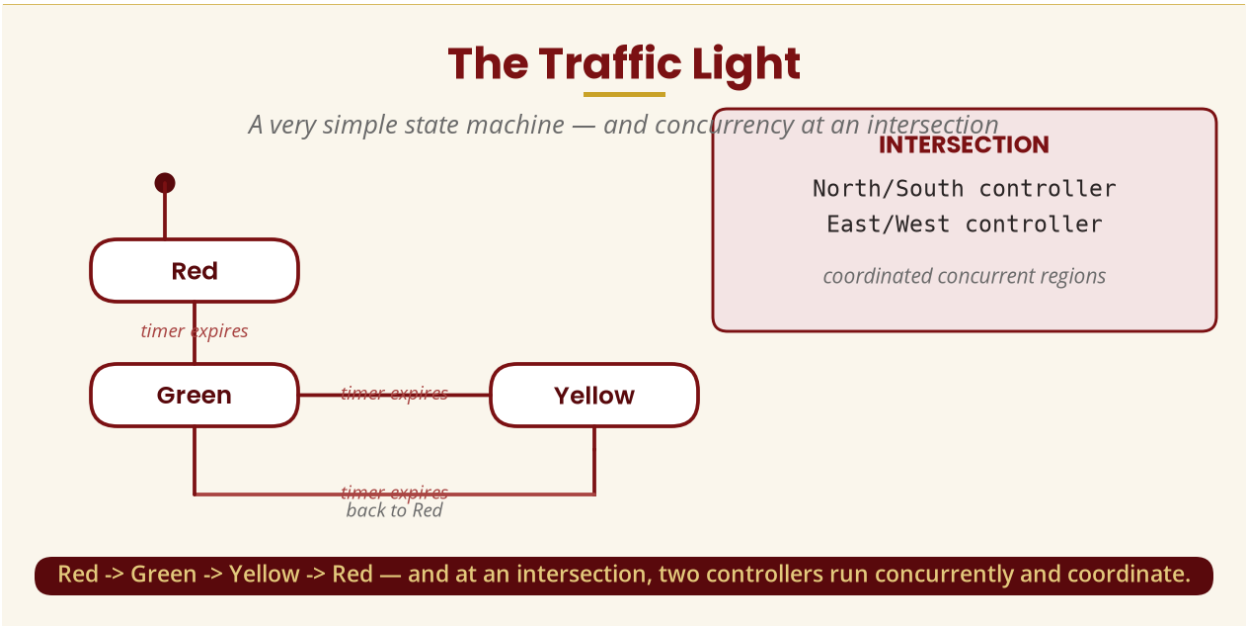

Figure 17 — Traffic light state machine and intersection concurrency

16. Practical Lab and Tutorial

Practical — Online Shopping Order: create a state-machine diagram with states New, Pending Payment, Paid, Processing, Shipped, Delivered, Cancelled and Returned; events placeOrder, makePayment, paymentConfirmed, dispatchOrder, deliverOrder, cancelOrder, requestReturn, approveReturn; add guards such as `requestReturn [within return period]`; and add entry behaviour such as `entry / sendNotification()` to Paid and `entry / notifyCustomer()` to Shipped.

Tutorial: (1) a bank account with states Active, Blocked, Closed and events login, wrongPIN, blockAccount, unblockAccount, closeAccount — draw the machine, add guards, and identify which transitions should not be allowed; (2) explain the difference between an event and a state; (3) explain why a class diagram alone is insufficient for a complex Order object.

17. Common Student Mistakes

Mistake 1 — Treating an event as a state.

"PaymentReceived" is an instantaneous event, not a state — model Pending □□PaymentReceived□□□ Paid.

Mistake 2 — Confusing an operation with a state.

processPayment() is an operation — model [Pending] □□processPayment()□□□ [Processing].

Mistake 3 — Forgetting the object being modelled.

A state machine should have a clear context (Order, Payment, ATM, StudentRegistration) — not a collection of random states.

Mistake 4 — Creating impossible transitions.

Delivered Paid may make no sense — use domain requirements, not arbitrary arrows.

Mistake 5 — A state for every small action.

“Click Button” is normally an event; “Processing” is a meaningful state.

18. Week 7 Summary and Key Terms

Concept	Simple meaning	Example
Dynamic modelling	Behaviour/change over time	Order lifecycle
Event	Something that happens	Payment received
State	Condition of an object	Paid
Transition	Movement between states	Pending Paid
Guard	Condition controlling a transition	[payment valid]
Operation	Behaviour/service of an object	cancelOrder()
Initial / final state	Starting point / end point	/
Entry / exit / do	Behaviour on entering / leaving / within a state	sendEmail()
Concurrency	Activities in parallel/overlapping	Payment + preparation
Structural / dynamic model	What exists / what happens	Classes / states

THE MOST IMPORTANT PICTURE

An object exists it is in a state an event occurs a transition happens the object enters another state. That is the heart of dynamic modelling. Whenever you face a dynamic-modelling problem, ask four questions: **What object am I modelling? What states can it be in? What events cause it to change? What conditions determine which transition occurs?** Answer those four questions and you are already thinking like an object-oriented analyst.

19. Examination-Style Questions

Question 1 (Definitions). Define dynamic modelling, event, state, transition, guard condition, operation and concurrency. [14 marks]

Question 2 (Differences). Explain the difference between: event and state; operation and event; structural model and dynamic model; sequential and concurrent behaviour. [16 marks]

Question 3 (State machine). A university student registration can move through Not Registered, Registration Pending, Payment Pending, Registered and Cancelled, with events submitRegistration, makePayment, paymentConfirmed and cancelRegistration. Draw an appropriate UML state-machine diagram. [15 marks]

Question 4 (Analysis). Explain why dynamic modelling is important when analysing an online banking system, including at least four examples of objects whose states change over time. [10 marks]

Question 5 (Concurrency). A food delivery application performs payment processing, restaurant order preparation, driver assignment and customer notification after an order is placed. Explain which activities could occur concurrently and why. [10 marks]

20. References and Further Reading

Primary authoritative reference

- Object Management Group. (2017). *OMG Unified Modeling Language (UML), Version 2.5.1.* OMG — the formal reference for UML, including the structural and behavioural semantic foundations and state machines.
- Object Management Group. *UML 2.5.1 — State Machines and Behavioural Semantics* (states, transitions, events, behaviours and related state-machine concepts).

Prescribed and recommended texts

- Mach, E. (2019). *Object Oriented Analysis & Design Cookbook: Introduction to Practical System Modeling.*
- Reddy, V., & Korra, S. (2018). *Object-Oriented Analysis and Design Using UML.*
- The Art of Service. (2021). *Object Oriented Analysis and Design: A Complete Guide.*
- Wixom, B., & Dennis, A. (2018). *Systems Analysis and Design.*
- Blokdyk, G. (2021). *Object-Oriented Analysis and Design Standard Requirements.*
- Wixom, B., & Dennis, A. (2020). *Systems Analysis and Design: An Object-Oriented Approach with UML.*